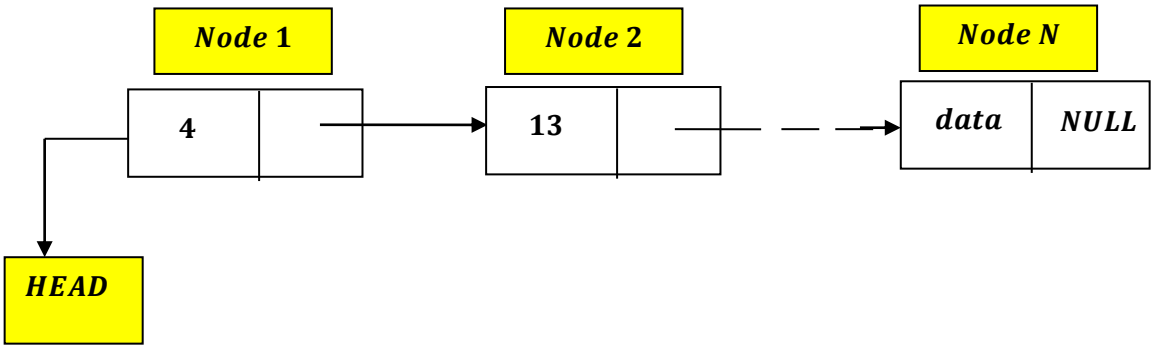
Linked – List – Introduction

What is a Linked List?

One disadvantage of using arrays to store data is that arrays are static structures and therefore cannot be easily extended or reduced to fit the data set. Arrays are also expensive to maintain new insertions and deletions. In this chapter we consider another data structure called Linked Lists that addresses some of the limitations of arrays.

A linked list is a data structure used for storing collections of data. A linked list has the following properties. A linked list is a linear dynamic data structure. The number of nodes in a list is not fixed and can grow and shrink on demand. Each node of a linked list is made up of two items – the data and a reference to the next node. The last node has a reference to null. The entry point into a linked list is called the head of the list. It should be noted that head is not a separate node, but the reference to the first node. If the list is empty then the head is a null reference.

- Successive elements are connected by pointers.
- The last element points to NULL.
- Can grow or shrink in size during execution of a program.
- Can be made just as long as required (until systems memory exhausts).
- Does not waste memory space (but takes some extra memory for pointers). It allocates memory as list grows.



Linked Lists ADT (Abstract Data Type)

The following operations make linked lists an ADT:

Main Linked Lists Operations:

- Insert: inserts an element into the list
- Delete: removes and returns the specified position element from the list.

Auxiliary Linked Lists Operations:

- Delete List: removes all elements of the list (disposes the list).
- Count: returns the number of elements in the list.
- Find n^{th} node from the end of the list.

Why Linked Lists?

There are many other data structures that do the same thing as linked lists. Before discussing linked lists, it is important to understand the difference between linked lists and arrays. Both linked lists and arrays are used to store collections of data, and since both are used for the same purpose, we need to differentiate their usage. That means in which cases arrays are suitable and in which cases linked lists are suitable.

Arrays Overview

One memory block is allocated for the entire array to hold the elements of the array. The array elements can be accessed in constant time by using the index of the particular element as the subscript.

	3	2	1	2	2	3
Index ←	0	1	2	3	4	5

Why Constant Time for Accessing Array Elements?

To access an array element, the address of an element is computed as an offset from the base address of the array and one multiplication is needed to compute what is supposed to be added to the base address to get the memory address of the element. First the size of an element of that data type is calculated and then it is multiplied with the index of the element to get the value to be added to the base address. This process takes one multiplication and one addition. Since these two operations take constant time, we can say the array access can be performed in constant time.

Advantages of Arrays

- *Simple and easy to use*
- *Faster access to the elements (constant access)*

Disadvantages of Arrays

- ***Preallocates all needed memory up front and wastes memory space for indices in the array that are empty***.

- ***Fixed size***: *The size of the array is static (specify the array size before using it)*.

- ***One block allocation***: *To allocate the array itself at the beginning, sometimes it may not be possible to get the memory for the complete array (if the array size is big)*.

- ***Complex position-****based insertion*:

To insert an element at a given position, we may need to shift the existing elements. This will create a position for us to insert the new element at the desired position. If the position at which we want to add an element is at the beginning, then the shifting operation is more expensive.

Dynamic Arrays

Dynamic array (also called as growable array, resizable array, dynamic table, or array list) is a random access, variable – size list data structure that allows elements to be added or removed.

One simple way of implementing dynamic arrays is to initially start with some fixed size array. As soon as that array becomes full, create the new array double the size of the original array. Similarly, reduce the array size to half if the elements in the array are less than half.

Note: We will see the implementation for dynamic arrays in the Stacks, Queues and Hashing chapters.

Advantages of Linked Lists

Linked lists have both advantages and disadvantages. The advantage of linked lists is that they can be expanded in constant time. To create an array, we must allocate memory for a certain number of elements. To add more elements to the array when full, we must create a new array and copy the old array into the new array. This can take a lot of time.

We can prevent this by allocating lots of space initially but then we might allocate more than we need and waste memory. With a linked list, we can start with space for just one allocated element and add on new elements

easily without the need to do any copying and reallocating.

Issues with Linked Lists (Disadvantages)

There are a number of issues with linked lists.
The main disadvantage of linked lists is access time to individual elements. Array is random − access, which means it takes $O(1)$ to access any element in the array. Linked lists take $O(n)$ for access to an element in the list in the worst case. Another advantage of arrays in access time is spacial locality in memory. Arrays are defined as contiguous blocks of memory, and so any array element will be physically near its neighbors. This greatly benefits from modern CPU caching methods.

Although the dynamic allocation of storage is a great advantage, the overhead with storing and retrieving data can make a big difference. Sometimes linked lists are hard to manipulate. If the last item is deleted, the last but one must then have its pointer changed to hold a NULL reference. This requires that the list is traversed to find the last but one link, and its pointer set to a NULL reference. Finally, linked lists waste memory in terms of extra reference points.

Comparison of Linked Lists with Arrays and Dynamic Arrays

As with most choices in computer programming and design, no method is well suited to all circumstances. A linked list data structure might ork well in one case, but cause problems in another. This is a list of some of the common trade – offs involving linked list structures. In general, you have a dynamic collection, where elements are frequently being added and deleted, and the location of new elements added to the significant, then benefits of a linked list increase.

Parameter	Linked list	Array	Dynamic array
Indexing	O(n)	0(1)	0(1)
Insertion/deletion at beginning	0(1)	O(n), if array is not full (for shifting the elements)	O(n)
Insertion at ending	O(n)	O(1), if array is not full	O(1), if array is not full O(n), if array is full
Deletion at ending	O(n)	0(1)	O(n)
Insertion in middle	O(n)	O(n), if array is not full (for shifting the elements)	O(n)
Deletion in middle	O(n)	O(n), if array is not full (for shifting the elements)	O(n)
Wasted space	(n) (for pointers)	0	O(n)

Feature	Array	Linked List
Memory Allocation	Contiguous	Non-contiguous
Size	Fixed	Dynamic
Access Time	O(1)[$We\ just\ use$: Address $=$ Base $+$ (index $\times$ size)$like: Arr[5]$]	O(n)[$Traversal\ from\ Head\ to\ Specific\ Node$]
Insertion	Costly	Efficient
Memory Usage	Less overhead	Extra pointer
Cache Performance	Good	Poor

****